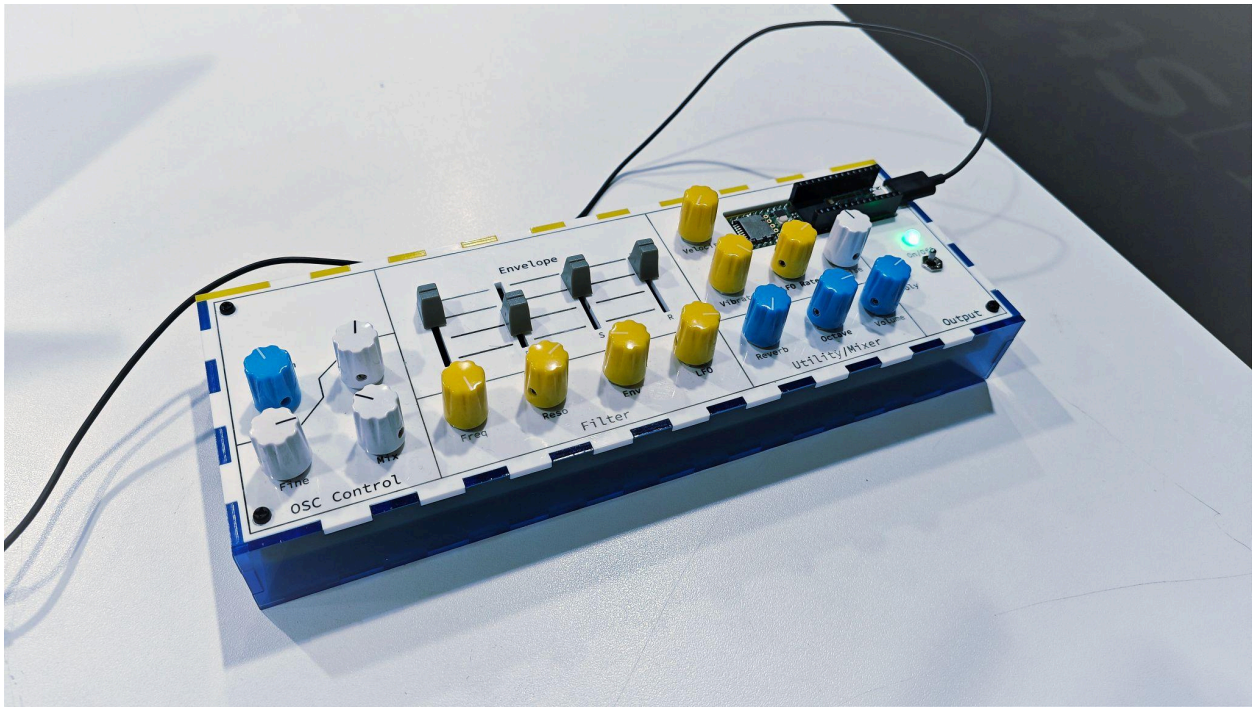
Music Technology Capstone Spring 2024

Building a Digital Guitar-to-Synth System Using Max and JUCE Framework

Michael Xie

Advisor: Prof.Paul Geluso & Prof.Steven Litt

GitHub Link: ZYFXzz/24_capstone (github.com) All related video links, documentation, and pictures will be saved in this repository.



Introduction

This capstone project embarked on the journey of creating a software-based system akin to renowned guitar pedal effects such as the Korg Miku Stomp, Boss SY-1, and Mooer E7 Synth. The primary aim was to develop a versatile system that could transform guitar inputs into compelling synth sounds, harnessing the power of digital signal processing, programming, and electronics—fields that dovetail with my academic pursuits and personal interests in guitar playing and Vocaloid.

Background

The project was initiated to mimic the Korg Miku Stomp pedal, a pedal effect released around 2014, capable of inputting guitar signals and outputting some human-like singing of Miku, lyrics, or simple vowels. During the research process, two other commercial models were also found to be great inspirations, BOSS SY-1 and Mooer E7 Synth.

During the process of research, the sub-field around pitch detection/estimation is recognized. It has already been a large sub-topic in the field of digital signal processing. Another note about it is that pitch is a psychological term, and we define that as the fundamental frequency of a signal. The pitch we perceive is the fundamental frequency, so-called F0. Different methods. In this area, I have identified an algorithm largely used for different purposes. The Ying Algorithm. Widely used for many purposes in music and speech. One pitch detection library in Teensy-Arduino was based on this algorithm.

Alain de Cheveigne, and Hideki Kawahara. “Yin, a Fundamental Frequency Estimator for Speech and Music.” *Http://Audition.Ens.Fr/Adc/Pdf/2002_JASA_YIN.Pdf*, audition.ens.fr/adc/pdf/2002_JASA_YIN.pdf. Accessed 11 Dec. 2023.

For the other synthesis component, it is also a large research field. The development of avant-garde electronic music and recent computer music can also be seen as the development of new synthesis methods. There are already many successful methods used in plugins, yet new methods were developed with recent research. Discussion about these methods can be found in the great book, the Computer Music Tutorial by Curtis Roads. A large section is all about different methods of synthesis. During the summer, I went through most parts of this great book, it covers different methods, including wavetable, granular, concatenative, Frequency Modulation, and Amplitude Modulation.

Roads, Curtis. *The Computer Music Tutorial*. The MIT Press, 2023.

The great tutorial book I am currently looking at is the Code Your Own Synth. It offers a great introduction to audio programming and coding a synth.

Hollemans, Matthijs. *Code Your Own Synth Plug-Ins With C++ and JUCE*. Independently Published, 2023.

There is a huge community around Audio Programming. The team of JUCE held an annual conference on audio plugin development in the UK. Even big audio companies like KORG, Arturia, UVI, and Universal Audio, were using JUCE, as indicated by the JUCE website.

Not only do big companies publish plugins, but small individuals or groups may develop great plugins. Julijan Nikolic developed the Youlean loudness meter for KVR developer challenges in 2016, and the plugin turned out to be a great success.

Another handful of books around the theory of audio plugin are Designing Audio Effect Plugins in C++.

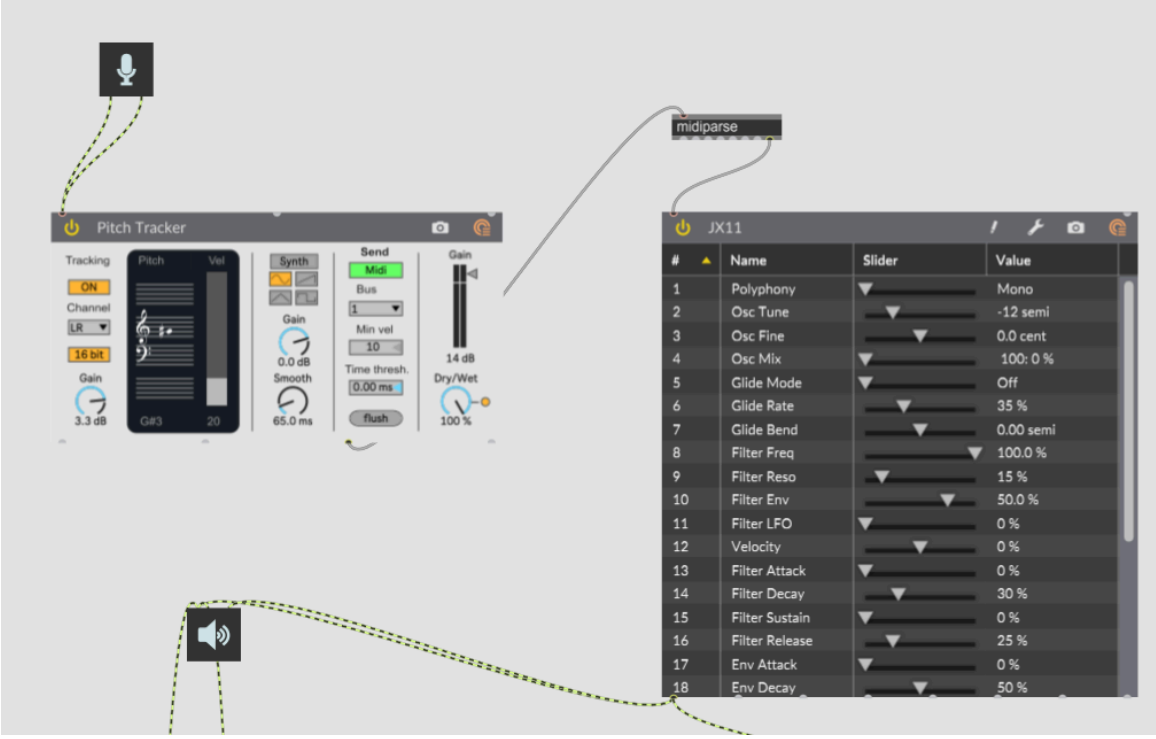
Pirkle, William C. *Designing Audio Effect Plugins in C++: For AAX, AU, and VST3 with DSP Theory*. Routledge, 2019.

To summarize, both F0 estimation and synthesis were already mature fields with plenty of ongoing development. Many great methods can be found around them.

Project Description

The designed system, implemented inside the max patch, features three core components: an F0 detector/estimator in max, a VST synth plugin as a re-synthesis module, and a hardware controller specifically designed for the plugin. Originally, more work should be put into the plugin build, but the time this semester is limited. The ideal outcome of the re-synthesis module is to output vocal samples. I hardly had enough time to explore and experiment with sample-based synthesis, so I gave up on the ideal goal. The VST currently in the system is a replica of the JX11 built by Hollemans. However, I have put more effort into designing the hardware midi interface for the plugin.

Max Patch



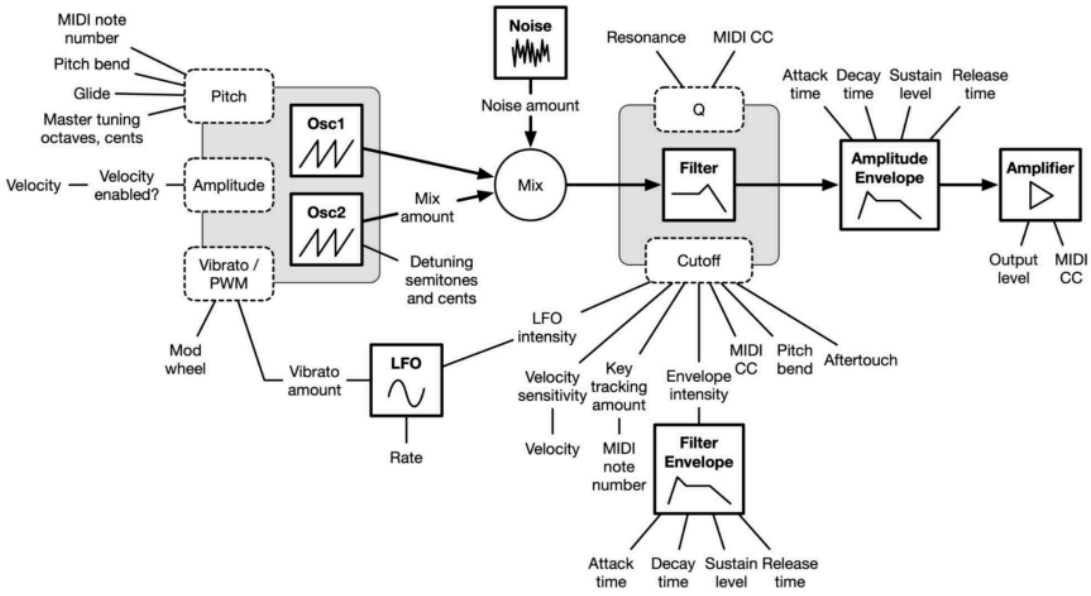
The current Max patch has two main modules, the pitch tracker provided by Max itself and the JX11 plugin as the synth core. The pitch tracker module will perform the task of transcribing audio into midi information.

JX11 vst plugin

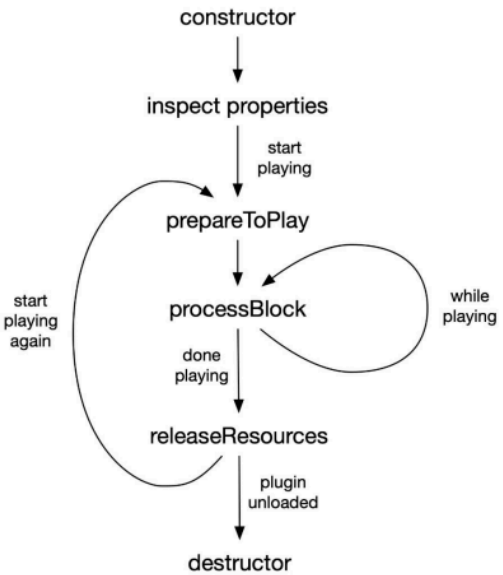
This is the synth plugin, which is the core focus of this project in the proposal. Ideally, this should developed into a sample-based synthesis, but I did not have enough time to explore that aspect. So basically it is a replica of the JX11 written by Hollemans. However, I still learn much from reading Holleman's tutorials and code. Such an experience is just a trial for audio programming, but it serves as a foundation for my future study in our music technology master program.

The JX11 introduced in the tutorial book is a digital version of an analog synth. It has two oscillators for sound generation, a filter, a simple LFO for modulation, and an ADSR envelope generator.

Here is the completed schematic of JX11 with all the pieces that make up the synth:



The full schematic of the synth



The lifecycle of a plug-in

The lifecycle of an audio plugin is also discussed in the book. It could be seen as an iterative process handling groups of audio samples. No matters it is a signal processing plugin like eq or compressor, or a synth/instrument plugin, this life cycle is the universal structure for all of them.

The Teensy Hardware MIDI controller

The hardware consists of mostly potentiometers, two switches, a neopixel digital LED, and one digital encoder. All are basic, common components that we introduced in electronics classes. With some product design knowledge learned from Prof.Litt, I put the prototype into a laser-cut box with basic interface designs.

The encoder is for selecting different preset programs of the plugin.

The potentiometer is for controlling the parameters of the plugin. To design a relatively small controller, I chose only part of the available parameters in all parameters available on JX11. The chosen parameters are the common parameters in an analog synth, such as oscillator tune, or ADSR envelope controls.

The other two switches act as simple mode selectors. 1 for polyphony or monophony selection. 1 for system bypass. The code behind them is the same as a basic pull-up push button.

The underlying Arduino code is the same for every potentiometer, which reads the analog voltage and turns it into a numerical value in 0-1023. Then this voltage reading will be turned into 0-126, a range for generic midi control value. The control value is later mapped onto the target parameter inside a daw. using the usbMIDI library provided along with the teensy board.

```
void potTurn(int number, int analogVal) {

  int a = map(analogVal, 1023, 0, 0, 126);

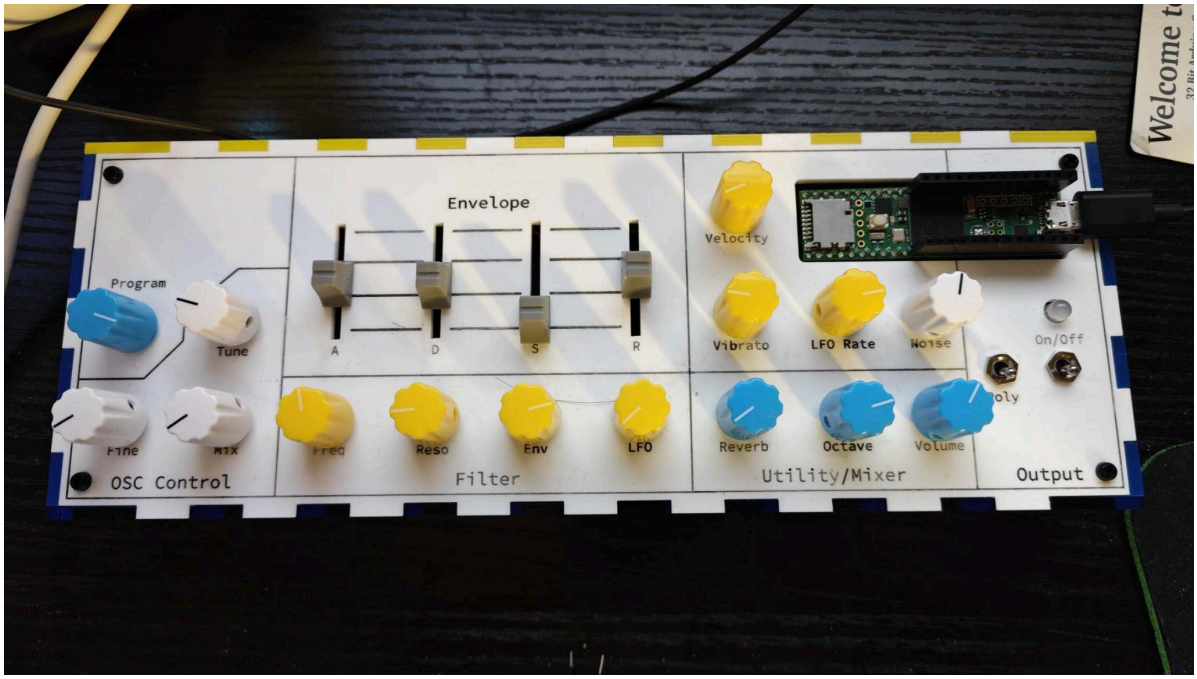
  usbMIDI.sendControlChange(potChannels[number], a, 1);

}
```

For the box design, I took inspiration from 2 identical synths we looked at in class EMS. The interface design is mostly borrowed from the MiniMoog Panel, and the ADSR sliders are borrowed from the ARP2600 design.

The pictures provided below are taken from the software version of the synths, products of Arturia.


Mini V3
MINI V
3osc*
Advanced
CONTROLLERS
OSCILLATOR BANK
MIXER
MODIFIERS
OUTPUT
ARTURIA


Envelope
Program
Tune
A
D
S
R
Velocity
Vibrato
LFO Rate
Noise
On/Off
Fine
Mix
Freq
Reso
Env
LFO
Reverb
Octave
Volume
OSC Control
Filter
Utility/Mixer
Output



Analysis and Conclusion

Throughout the development process, the system demonstrated a basic ability to handle real-time audio processing with some latency, but it is suitable for performing a melody in a slow tempo. Although pitch detection presented challenges, the use of a Max Patch for MIDI conversion and a custom-built synth plugin developed with C++ and JUCE proved effective. My video demonstration will include using my system and playing along with the demo video of the Korg pedal.

To me, this is a great conclusion to my coursework in the undergrad degree.

I also learned many things about audio programming. First is a general understanding of the architecture behind a vst plugin, this understanding will help to use those tools for my music better.

Another fact realized is that audio programming is very practical, and potentially the most practical topic in all programming tasks. The architecture of our current audio software is based on buffers. One important practical question for every audio programmer, especially for applications with real-time playback functions, is the tradeoff between sound quality and efficiency/latency. In common sense from a non-professional person, to construct a sine wave signal in the audio domain, we will calculate every sample using the standard sine function, and for other waveforms, we will use the correct, precise mathematical equation to calculate every sample. In practice, this is merely appreciated. Many current days' audio synthesis algorithms are a mere approximation of the true mathematical equation. If we

perform a null test, they will not be the same. However, such a tradeoff gives better efficiency, in which samples can be calculated using less computational resources and time. There are also many practical ideas similar to this, all are trading off some audio fidelity/precision for better efficiency. The design of those approximation algorithms usually also considered psychoacoustic effects and human ears hardly perceived the unnaturalness of the approximations.

The limitations of digital sampling are another area of problems that programmers have to deal with. For the JX11 sawtooth generator, the original design choose the BLIT algorithm to generate a band-limited sawtooth wave. Band limited means it is still far away from an ideal sawtooth wave, and an inaudible artifact for this band-limited sawtooth wave is a short DC offset for a few oscillating cycles around the note onset. However, this is still good as humans cannot perceive it.

Future Work

- **Improving the pitch-tracking module.** There are different tools for the task that may perform better than my current solutions. The current max pitch track module also has many noise. Many small midi chunks are also introduced while tracking the melody. May need some debouncing solutions to ensure a smooth re-synthesis sound. Also, the original Miku Korg Pedal can distinguish between a new note and vibrato produced by bending the strings. However, the max patch cannot distinguish between those, it will output a new midi on/off event when the pitch changes for more than 1 semitone.

- **Put more things inside the max patch.** Many midi effects in Max could be done. One simple example of extra work is receiving a single note from a guitar, but playing out a chord as the output. Another thing I learned from advanced computer music composition class is to use external AV sources to interactively control some parameters inside a max patch, we can use camera video sources to control synth parameters or MIDI effects.

- **Today's new audio transformation based on deep learning**: With today's machine learning being applied to the audio domain, people have fallen off their interest in traditional MIDI or DSP-based transforming systems. Researchers are now embracing novel transforming methods with today's deep-learning systems.

  One notable system is capable of transforming one type of sound to another type, for example changing a bird's voice to a violin playing, preserving some basic elements like F0 and note length which drastically changed the timbre. Those audio-to-audio DL models seem to open up the sound palette for sound/timbre transformation. This direction is worth looking at in the future.

References

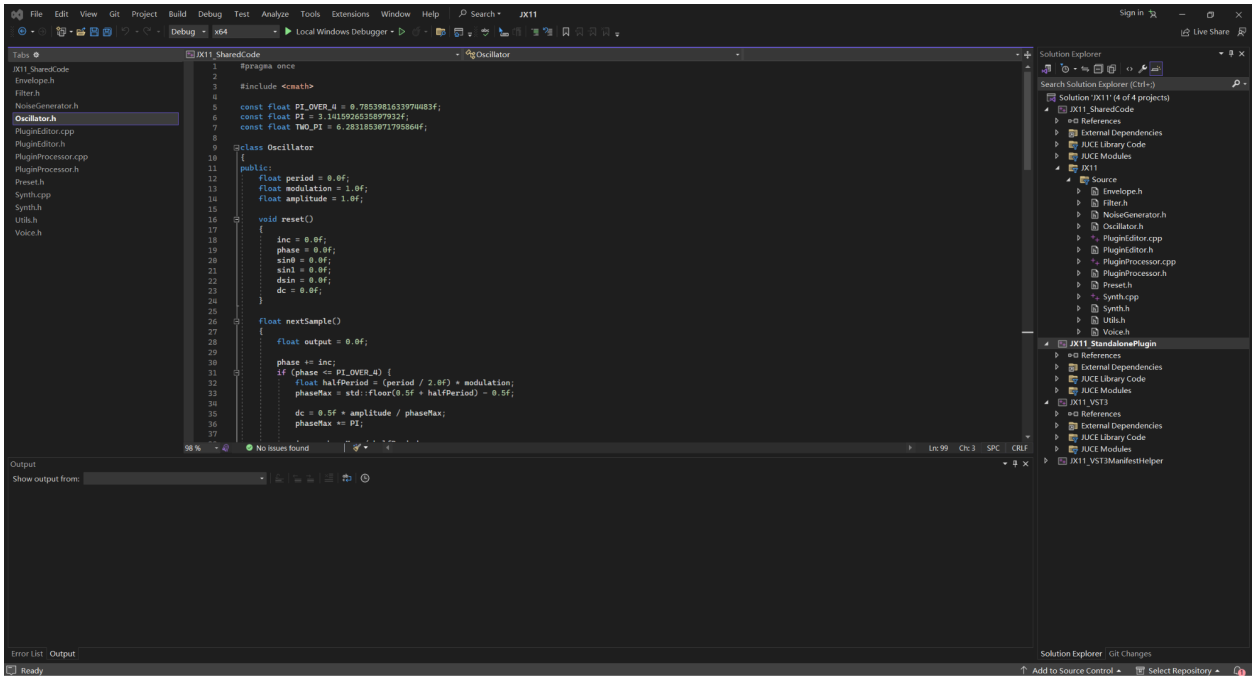
Alain de Cheveigne, and Hideki Kawahara. “Yin, a Fundamental Frequency Estimator for Speech and Music.” Http://Audition.Ens.Fr/Adc/Pdf/2002_JASA_YIN.Pdf, audition.ens.fr/adc/pdf/2002_JASA_YIN.pdf. Accessed 11 Dec. 2023.

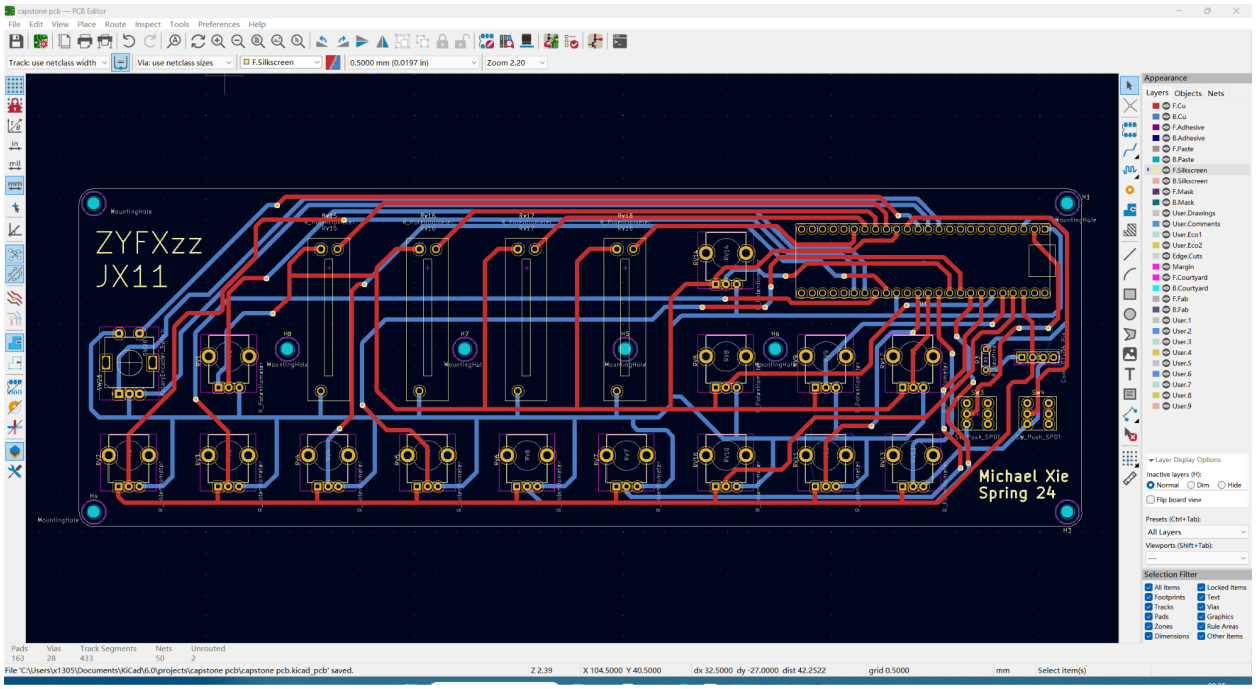
Roads, Curtis. The Computer Music Tutorial. The MIT Press, 2023.

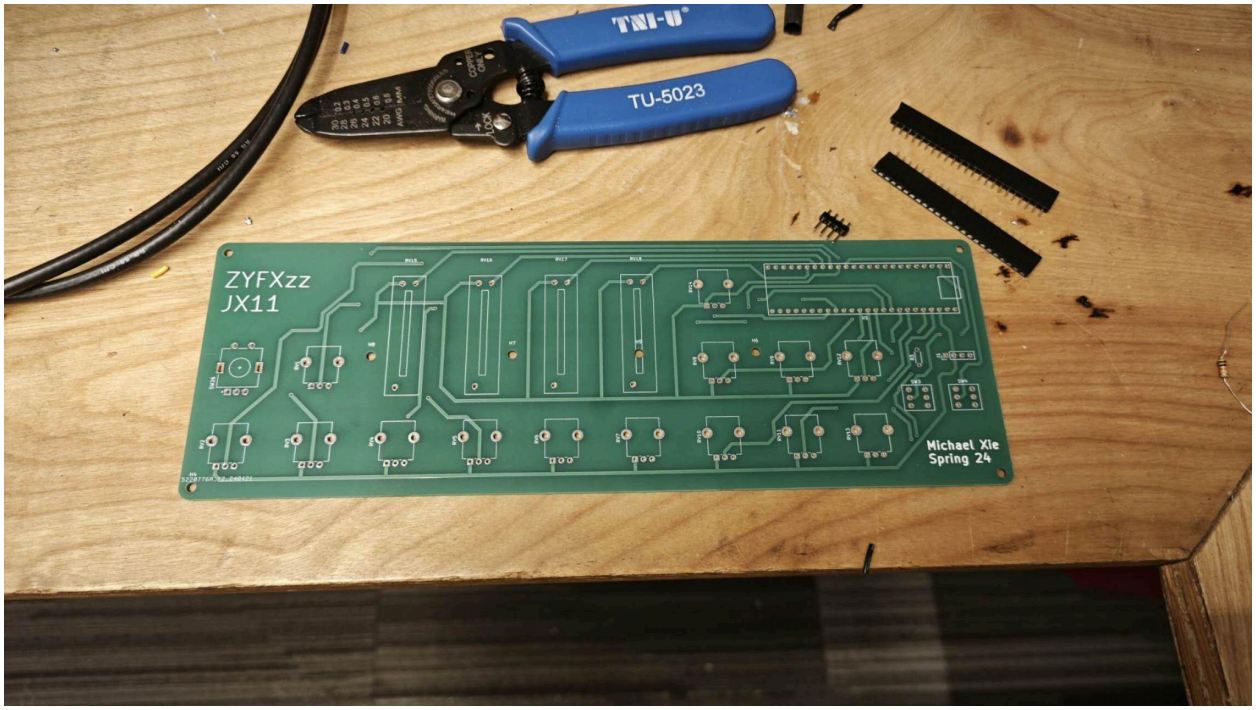
Hollemans, Matthijs. Code Your Own Synth Plug-Ins with C++ and JUCE. Independently Published, 2023.

Pirkle, William C. Designing Audio Effect Plugins in C++: For AAX, AU, and VST3 with DSP Theory. Routledge, 2019.

Other pictures in developing the project:


#pragma once
#include <cmath>
const float PI_OVER_4 = 0.78539816339744831f;
const float PI = 3.14159265358979323f;
const float TWO_PI = 6.28318530717958864f;
class Oscillator
{
public:
float period = 0.0f;
float modulation = 1.0f;
float amplitude = 1.0f;
void reset()
{
inc = 0.0f;
phase = 0.0f;
sin0 = 0.0f;
sin1 = 0.0f;
dsin = 0.0f;
dc = 0.0f;
}
float nextSample()
{
float output = 0.0f;
phase += inc;
if (phase <= PI_OVER_4) {
float halfPeriod = (period / 2.0f) * modulation;
phaseMax = std::floor(0.5f + halfPeriod) - 0.5f;
dc = 0.5f * amplitude / phaseMax;
phaseMax *= PI;


ZYFXzz
JX11
Michael Xie
Spring 24


TNI-U
TU-5023
ZYFXzz
JX11
Michael Xie
Spring 24